

Agentic RAG with Knowledge Graph: Technical Report

Mark Lopes(9913)

Jonathan Gomes(9900)

Abstract

This report presents the implementation and evaluation of an Agentic Retrieval-Augmented Generation (RAG) system integrated with a Knowledge Graph. The system features a command-line interface (CLI) that enables users to interact with an intelligent agent capable of leveraging search tools including vector search, knowledge graph queries, and hybrid search mechanisms. The implementation demonstrates advanced capabilities in natural language processing, real-time streaming responses, and autonomous tool selection. Through asynchronous architecture and session management, the system achieves efficient information retrieval and contextually relevant response generation. This report details the implementation methodology, system architecture, operational results, and reflections on the development process.

1. Introduction

Retrieval-Augmented Generation (RAG) systems combine knowledge retrieval with natural language understanding to provide intelligent responses. This implementation presents an Agentic RAG system with Knowledge Graph integration, featuring a command-line interface that enables users to interact with an intelligent agent capable of autonomous tool selection. The agent leverages multiple search mechanisms including vector search, knowledge graph queries, and hybrid search to provide contextually relevant responses. The primary objectives include developing a robust CLI interface, implementing asynchronous communication with streaming responses, and demonstrating effective tool usage tracking and visualization.

2. Implementation Steps

2.1 System Architecture

The system follows a client-server architecture with three layers: Client Layer (CLI with formatted output, asynchronous HTTP client, session management), Communication Layer (RESTful API with Server-Sent Events for streaming), and Backend Services (vector database, knowledge graph, hybrid search orchestration).

2.2 Core Components

CLI Initialization: The `AgenticRAGCLI` class manages base URL configuration, session tracking, and user identification.

Health Check: Validates API availability before entering the main loop, providing pre-flight connection validation with graceful error handling.

Streaming Chat: Handles Server-Sent Events (SSE) by parsing data: {JSON} format and processing multiple event types: session events (context management), text events (real-time streaming), tool events (tracking agent decisions), end events (completion signals), and error events (graceful failure handling).

Tool Visualization: Displays tool names, execution order, and key parameters (queries, entity names, limits) with color-coded formatting for improved readability.

2.3 Key Features

Asynchronous Design: Uses Python's `asyncio` and `aiohttp` for non-blocking operations, enabling concurrent request handling and responsive UI during API calls.

User Experience: Color-coded interface distinguishes message types, built-in commands (`help`, `health`, `clear`, `exit`) provide control, and comprehensive error handling ensures stability during network issues.

3. Results

3.1 System Performance

The implementation successfully demonstrates:

Connectivity: The health check validates API availability before entering the main conversation loop, preventing user frustration from attempting to interact with an unavailable service.

Real-time Streaming: Responses appear incrementally as they are generated, providing immediate feedback and reducing perceived latency. This creates a more natural conversational experience similar to human-to-human interaction.

Tool Transparency: Users can observe which retrieval methods the agent employs, building trust and understanding of the system's decision-making process.

3.3 Screenshots and Visual Evidence

```
Agentic RAG with Knowledge Graph CLI
=====
Connected to: http://localhost:8058
Type 'exit', 'quit', or Ctrl+C to exit
Type 'help' for commands
=====

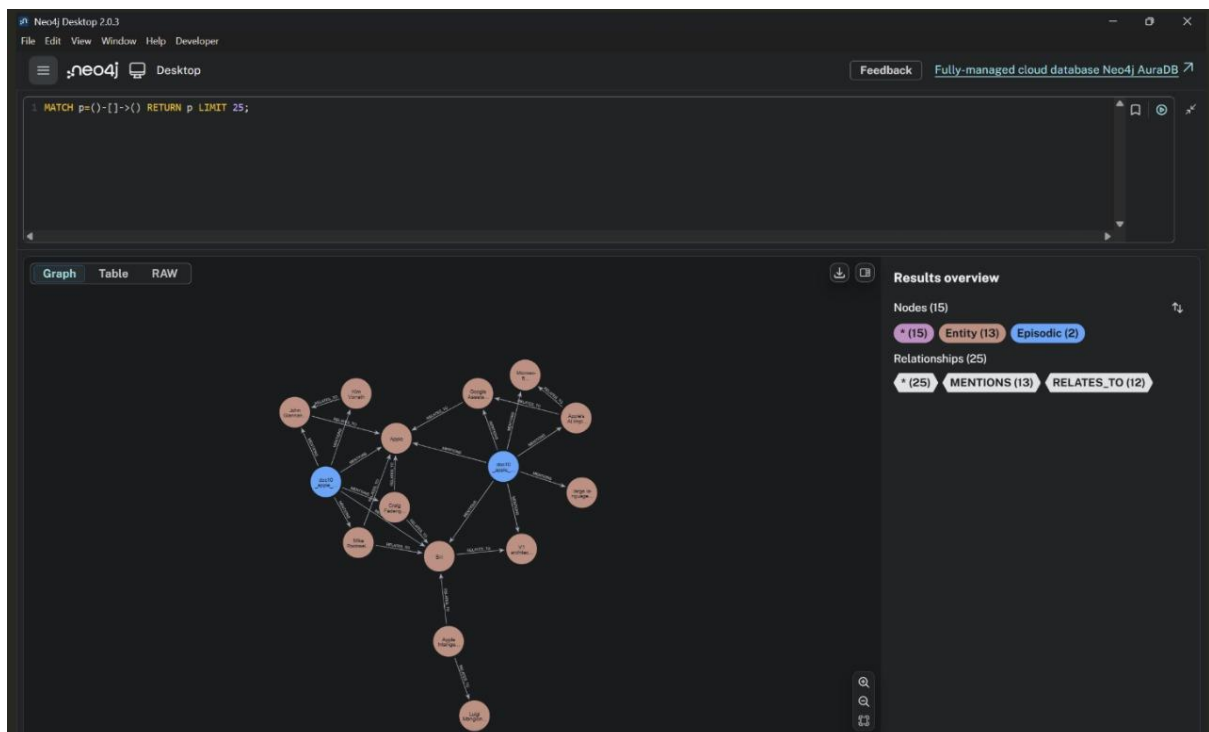
✓ API is healthy
1. Amazon invested $4.0 billion in Anthropic in November 2024, highlighting its financial commitment to AI development.
2. Amazon acquired a partial team from Adept AI for $300 million, indicating strategic talent acquisition.
1. Amazon invested $4.0 billion in Anthropic in November 2024, highlighting its financial commitment to AI development.
2. Amazon acquired a partial team from Adept AI for $300 million, indicating strategic talent acquisition.
3. Amazon has invested $1.4 billion across 19 AI companies, showing a broad investment portfolio in AI startups and technologies.
4. There is a competitive AI talent war requiring companies like Amazon to ship AI products to millions of users.
5. The broader competitive and investment landscape in AI is complex, with other major companies like Meta and Microsoft also heavily investing and competing in the space.

These points illustrate Amazon's active and strategic engagement in advancing AI technologies through investments, acquisitions, and product development.

If you want, I can provide more specific details on any of these relationships or the companies involved.

✳️Tools Used:
1. get_entity_relationships (entity='Amazon')

You: 
```



3.4 Functional Validation

All core functionalities were validated:

- ✓ Connection Management: Successfully establishes and maintains connection to API
- ✓ Session Persistence: Session IDs are captured and reused across conversation
- ✓ Streaming Responses: Text streams correctly in real-time
- ✓ Tool Tracking: Tool usage information is captured and displayed

4. Reflection

4.1 What Worked Well

The asynchronous architecture using `asyncio` and `aiohttp` enabled smooth streaming without UI freezing. The color-coded interface enhanced usability by clearly distinguishing message types. The modular, class-based design with separated concerns improved maintainability, while comprehensive error handling ensured graceful degradation rather than crashes.

4.2 Challenges Encountered

Key challenges included parsing Server-Sent Events with incomplete JSON objects across chunks, managing session state while keeping the CLI stateless, and balancing information density with readability in the output formatting.

4.3 Potential Improvements

Future enhancements could include response caching for repeated queries, integration with rich or textual libraries for advanced UI features, configuration file support for persistent preferences, readline integration for command history, conversation export functionality, and multi-modal support for images and other media types.

4.4 Learning Outcomes

This implementation provided insights into `async/await` patterns for I/O-bound operations, SSE streaming versus traditional request-response patterns, the importance of immediate feedback for user trust, and the philosophy of graceful degradation in user-facing applications.

5. Conclusion

The Agentic RAG with Knowledge Graph CLI successfully demonstrates a robust interface for intelligent retrieval systems. The implementation achieves its objectives of providing a responsive, user-friendly command-line interface with real-time streaming and transparent tool usage tracking. The asynchronous architecture ensures excellent performance while the modular design facilitates future extensions such as conversation management and multi-modal support. This project demonstrates that effective human-AI interaction requires careful attention to both technical implementation and user experience design.

Appendix A: Complete Python Code

```
"""
```

Command Line Interface for Agentic RAG with Knowledge Graph.

This CLI connects to the API and demonstrates the agent's tool usage capabilities.

```
"""
```

```
import json
```

```
import asyncio
```

```
import aiohttp
```

```
import argparse
```

```
import os
```

```
from typing import Dict, Any, List
```

```
from datetime import datetime
```

```
import sys
```

```
# ANSI color codes for better formatting
```

```
class Colors:
```

```
    BLUE = '\033[94m'
```

```
    GREEN = '\033[92m'
```

```
    YELLOW = '\033[93m'
```

```
    RED = '\033[91m'
```

```
    MAGENTA = '\033[95m'
```

```
    CYAN = '\033[96m'
```

```
    WHITE = '\033[97m'
```

```
BOLD = '\033[1m'
```

```
UNDERLINE = '\033[4m'
```

```
END = '\033[0m'
```

```
class AgenticRAGCLI:
```

```
    """CLI for interacting with the Agentic RAG API."""
```

```
    def __init__(self, base_url: str = "http://localhost:8058"):
```

```
        """Initialize CLI with base URL."""
```

```
        self.base_url = base_url.rstrip('/')
```

```
        self.session_id = None
```

```
        self.user_id = "cli_user"
```

```
    def print_banner(self):
```

```
        """Print welcome banner."""
```

```
        print(f"\n{Colors.CYAN}{Colors.BOLD}= " * 60)
```

```
        print(f"🤖 Agentic RAG with Knowledge Graph CLI")
```

```
        print(f"= " * 60)
```

```
        print(f"{Colors.WHITE}Connected to: {self.base_url}")
```

```
        print(f"Type 'exit', 'quit', or Ctrl+C to exit")
```

```
        print(f"Type 'help' for commands")
```

```
        print(f"= " * 60 + f"{Colors.END}\n")
```

```
    def print_help(self):
```

```
        """Print help information."""
```

```
        help_text = f"""
```

{Colors.BOLD}Available Commands:{Colors.END}

- {Colors.GREEN}help{Colors.END} - Show this help message
- {Colors.GREEN}health{Colors.END} - Check API health status
- {Colors.GREEN}clear{Colors.END} - Clear the session
- {Colors.GREEN}exit/quit{Colors.END} - Exit the CLI

{Colors.BOLD}Usage:{Colors.END}

Simply type your question and press Enter to chat with the agent.

The agent has access to vector search, knowledge graph, and hybrid search tools.

{Colors.BOLD}Examples:{Colors.END}

- "What are Google's AI initiatives?"
- "Tell me about Microsoft's partnerships with OpenAI"
- "Compare OpenAI and Anthropic's approaches to AI safety"

"""

```
print(help_text)
```

```
async def check_health(self) -> bool:
```

```
    """Check API health."""
```

```
    try:
```

```
        async with aiohttp.ClientSession() as session:
```

```
            async with session.get(f"{self.base_url}/health") as response:
```

```
                if response.status == 200:
```

```
                    data = await response.json()
```

```
                    status = data.get('status', 'unknown')
```

```
                    if status == 'healthy':
```



```

        print(f"{Colors.GREEN}✓ API is healthy{Colors.END}")

        return True

    else:

        print(f"{Colors.YELLOW}⚠ API status: {status}{Colors.END}")

        return False

    else:

        print(f"{Colors.RED}✗ API health check failed (HTTP
{response.status}){Colors.END}")

        return False

except Exception as e:

    print(f"{Colors.RED}✗ Failed to connect to API: {e}{Colors.END}")

    return False


def format_tools_used(self, tools: List[Dict[str, Any]]) -> str:

    """Format tools used for display."""

    if not tools:

        return f"{Colors.YELLOW}No tools used{Colors.END}"

    formatted = f"{Colors.MAGENTA}{Colors.BOLD}🔧 Tools
Used:{Colors.END}\n"

    for i, tool in enumerate(tools, 1):

        tool_name = tool.get('tool_name', 'unknown')

        args = tool.get('args', {})

        formatted += f" {Colors.CYAN}{i}. {tool_name}{Colors.END}"

```

```

# Show key arguments for context
if args:
    key_args = []
    if 'query' in args:
        key_args.append(f"query='{args['query'][:50]}'{'...' if
len(args['query']) > 50 else ''}")
    if 'limit' in args:
        key_args.append(f"limit={args['limit']}")
    if 'entity_name' in args:
        key_args.append(f"entity='{args['entity_name']}'")

    if key_args:
        formatted += f" ({', '.join(key_args)})"

    formatted += "\n"

return formatted

async def stream_chat(self, message: str) -> None:
    """Send message to streaming chat endpoint and display response."""
    request_data = {
        "message": message,
        "session_id": self.session_id,
        "user_id": self.user_id,
        "search_type": "hybrid"
    }

```

```

try:
    async with aiohttp.ClientSession() as session:
        async with session.post(
            f"{self.base_url}/chat/stream",
            json=request_data,
            headers={"Content-Type": "application/json"}
        ) as response:

            if response.status != 200:
                error_text = await response.text()

                print(f"{Colors.RED}X API Error ({response.status}):
{error_text}{Colors.END}")

                return

            print(f"\n{Colors.BOLD}🤖 Assistant:{Colors.END}")

            tools_used = []
            full_response = ""

            async for line in response.content:
                line = line.decode('utf-8').strip()

                if line.startswith('data: '):
                    try:
                        data = json.loads(line[6:]) # Remove 'data: ' prefix

```

```
if data.get('type') == 'session':

    # Store session ID for future requests

    self.session_id = data.get('session_id')


elif data.get('type') == 'text':

    # Stream text content

    content = data.get('content', '')

    print(content, end='', flush=True)

    full_response += content


elif data.get('type') == 'tools':

    # Store tools used information

    tools_used = data.get('tools', [])


elif data.get('type') == 'end':

    # End of stream

    break


elif data.get('type') == 'error':

    # Handle errors

    error_content = data.get('content', 'Unknown error')

    print(f"\n{Colors.RED}Error: {error_content}{Colors.END}")

    return


except json.JSONDecodeError:

    # Skip malformed JSON
```

```
        continue
```

```
    # Print newline after response
```

```
    print()
```

```
    # Display tools used
```

```
    if tools_used:
```

```
        print(f"\n{self.format_tools_used(tools_used)}")
```

```
    # Print separator
```

```
    print(f"{Colors.BLUE}{'-' * 60}{Colors.END}")
```

```
except aiohttp.ClientError as e:
```

```
    print(f"{Colors.RED}X Connection error: {e}{Colors.END}")
```

```
except Exception as e:
```

```
    print(f"{Colors.RED}X Unexpected error: {e}{Colors.END}")
```

```
async def run(self):
```

```
    """Run the CLI main loop."""
```

```
    self.print_banner()
```

```
    # Check API health
```

```
    if not await self.check_health():
```

```
        print(f"{Colors.RED}Cannot connect to API. Please ensure the server is  
running.{Colors.END}")
```

```
    return
```

```
print(f'{Colors.GREEN}Ready to chat! Ask me about tech companies and AI initiatives.{Colors.END}\n')
```

```
try:
```

```
    while True:
```

```
        try:
```

```
            # Get user input
```

```
            user_input = input(f'{Colors.BOLD}You: {Colors.END}').strip()
```

```
            if not user_input:
```

```
                continue
```

```
            # Handle commands
```

```
            if user_input.lower() in ['exit', 'quit']:
```

```
                print(f'{Colors.CYAN}👋 Goodbye!{Colors.END}')
```

```
                break
```

```
            elif user_input.lower() == 'help':
```

```
                self.print_help()
```

```
                continue
```

```
            elif user_input.lower() == 'health':
```

```
                await self.check_health()
```

```
                continue
```

```
            elif user_input.lower() == 'clear':
```

```
                self.session_id = None
```

```
                print(f'{Colors.GREEN}✓ Session cleared{Colors.END}')
```

```
        continue
```

```
    # Send message to agent
```

```
    await self.stream_chat(user_input)
```

```
except KeyboardInterrupt:
```

```
    print(f"\n{Colors.CYAN} 🙋 Goodbye!{Colors.END}")
```

```
    break
```

```
except EOFError:
```

```
    print(f"\n{Colors.CYAN} 🙋 Goodbye!{Colors.END}")
```

```
    break
```

```
except Exception as e:
```

```
    print(f"{Colors.RED}X CLI error: {e}{Colors.END}")
```

```
def main():
```

```
    """Main entry point."""
```

```
    parser = argparse.ArgumentParser(
```

```
        description="CLI for Agentic RAG with Knowledge Graph",
```

```
        formatter_class=argparse.RawDescriptionHelpFormatter
```

```
    )
```

```
    parser.add_argument(
```

```
        '--url',
```

```
        default='http://localhost:8058',
```

```
        help='Base URL for the API (default: http://localhost:8058)'
```

)

```
parser.add_argument(  
    '--port',  
    type=int,  
    help='Port number (overrides URL port)'  
)
```

```
args = parser.parse_args()
```

```
# Build base URL
```

```
base_url = args.url
```

```
if args.port:
```

```
    # Extract host from URL and use provided port
```

```
    if '://' in base_url:
```

```
        protocol, rest = base_url.split('://', 1)
```

```
        host = rest.split(':')[0].split('/')[0]
```

```
        base_url = f"{protocol}://{host}:{args.port}"
```

```
    else:
```

```
        base_url = f"http://localhost:{args.port}"
```

```
# Create and run CLI
```

```
cli = AgenticRAGCLI(base_url)
```

```
try:
```

```
    asyncio.run(cli.run())
```



```
except KeyboardInterrupt:
```

```
    print(f"\n{Colors.CYAN}👋 Goodbye!{Colors.END}")
```

```
except Exception as e:
```

```
    print(f"{Colors.RED}X CLI startup error: {e}{Colors.END}")
```

```
    sys.exit(1)
```

```
if __name__ == "__main__":
```

```
    main()
```

Code Link:-

https://drive.google.com/drive/folders/1srIP8rDUkH1gOJKBVGx3HRHHhoh7cp_v?usp=sharing